第3章　类与对象

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

C中结构体

```
int x;
float y;

struct Student{
    unsigned id;
    char name[20];
    float height;
    float weight;
};
Student  s;
s.id = 100;
printf("%s\n", s.name);
IsHealthy(s.height, s.weight);
```

C中的结构体

✓int、float、Student：数据类型
✓x、y、s：变量

✓结构体类型将多个相关变量包装为一个整体使用。

✓C中的结构体：数据与算法分离

C++中结构体

```
struct Student{
    unsigned id;
    char name[20];
    float height;
    float weight;
    bool IsHealthy(){…}
};
Student s;
s.IsHealthy();

s.id = 100;
printf("%s\n", s.name);
IsHealthy(s.height, s.weight);
```

C++中的结构体

✓C的结构不含函数。

✓在C++中，结构体除了可以包含变量外，还可以包含函数。

✓C++中的结构体：数据与算法组合

✓结构体中的成员变量可以被任意修改

第三章 类与对象

~~1.结构体与类~~
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

类

```
class Student{
    unsigned id;
    char name[20];
    float height;
    float weight;
    bool IsHealthy(){…}
};
Student s;
s.IsHealthy();

s.id = 100;
printf("%s\n", s.name);
IsHealthy(s.height, s.weight);
```

✓Student：类(类名首字母一般大写)
✓s：对象

✓类的结尾加分号“；”（同结构体）

✓类中的成员变量和成员函数默认都不能访问（封装）

✓【OO基本特征】封装、继承、多态

类

```
class Student{
public:
    bool IsHealthy(){…}
private:
    unsigned id;
    char name[20];
    float height;
    float weight;
};
Student s;
s.IsHealthy();
s.id = 100;
printf("%s\n", s.name);
IsHealthy(s.height, s.weight);
```

✓类中不允许外部访问的成员（数据、函数）设置为私有private

✓类中允许外部访问的成员（数据、函数）设置为公有public

✓【类中只允许内部和子类（继承）内部访问的成员设置成类型protected】

✓结构体内默认公有public，类内默认私有private

✓公有函数在前，私有函数、数据在后

类

```
class Student{
public:
    bool IsHealthy(){…}
    void SetId(int inputId)
      {id = inputId;}
    void PrintName()
     {printf("%s\n",name); }
private:
    unsigned id;
    char name[20];
    float height;
    float weight;
};
```

```
Student s;
s.IsHealthy();
s.SetId(100);
s.PrintName();
```

类的定义

```
class Student{
public:
    bool IsHealthy(){…}
    void SetId(int inputId){…}
    void PrintName(){…};
private:
    unsigned id;
    char name[20];
    float height;
    float weight;
};

Student s1, s2,s3;
s1.IsHealthy();
s2.SetId(100);
s3.PrintName();
```

类(class) ——数据类型

✓**类**是具有相同属性和方法的对象的集合，它是一种既包含数据又包含函数的抽象数据类型。

✓**类的名称**是一个合法的C++标识符，在程序中唯一的标识一个类。

✓**类的构成**

（1）数据成员（Data Member），即类的属性，代表类的静态特征；

（2）成员函数（Member Function），即类的方法，代表类的动态特性，也就是类的方法或行为，用来对数据成员进行操作 。类的成员函数同普通函数在语法上几乎相同，不同的是它只属于一个类，要调用一个类的成员函数，必须给出该类的对象名。

类的实现

```
class Student{
public:
    bool IsHealthy(){…}
    void SetId(int inputId){…}
    void PrintName();
private:
    unsigned id;
    char name[20];
    float height;
    float weight;
};
void Student::PrintName(){…};

Student s1, s2,s3;
s1.IsHealthy();
s2.SetId(100);
s3.PrintName();
```

类的定义格式

一般包含如下两个部分：

一、类定义

class 类名 *// class为类说明的关键字*
{
[public:] | [protected:] | [private:]
成员函数;
[public:] | [protected:] | [private:]
数据成员;
}; *//类体*

二、类的实现

各成员函数的定义或实现

✓ **内部定义：函数声明+定义（内联函数）：函数短、高效**
IsHealthy()、SetId()

✓ **外部定义：内部声明+外部定义**
PrintName()

类的使用

Student.h

```
class Student{
public:
    bool IsHealthy();
    void SetId(int);
    void PrintName();
private:
    unsigned id;
    char name[20];
    float height;
    float weight;
};
```

main.cpp

```
#include "Student.h"
Student s;
s.IsHealthy();
```

Student.cpp

```
#include "Student.h"
void Student::IsHealthy(){…}
void Student::SetId(int inputId){…}
void Student::PrintName(){…}
```

✓ **类的定义应该放在所有使用该类声明对象的程序段的前面**

✓ **一般地，将一个类的定义放在单独的一个头文件中，类的实现放在对应的cpp（c plus plus）文件中**

✓ **这样，在任何需要使用该类声明对象的源程序文件中，只要是用#include语句将含有这个类的定义的头文件包含进来即可。**

作用域限定符

```
void Set(int m, int d, int y);

class Tdate
{
  public:
    void Set(int m, int d, int y);
};

void Tdate::Set(int m, int d, int y)
{
    ::Set(m, d, t); //调用非成员函数
}
```

➢ **::** ——作用域限定符
- ➢指明一个函数、数据属于哪个类
- ➢**::** 可以不跟类名，表示全局数据或全局函数（即非成员函数）

注意事项1—类定义中不能初始化

Student.h

```
class Student{
public:
    bool IsHealthy();
    void SetId(int);
    void PrintName();
private:
    unsigned id = 100;
    char name[20];
    float height;
    float weight;
} ;

int =100;
```

✓在类内**不允许**对所定义的数据成员进行初始化，因为类是一个数据类型，其定义后并没有存储空间，因此，数据成员就不能进行初始化。

注意事项2—类的成员函数可以重载

Student.h

```
class Student{
public:
    bool IsHealthy();
    void SetId(int);
    void PrintName();
    void PrintName(bool);
private:
    unsigned id;
    char name[20];
    float height;
    float weight;
} ;
```

✓类的成员函数可以与C++传统函数一样的方法重载。但是类名不同就不是重载。

注意事项3—访问修饰符

```
class Student{
private:
    unsigned id;
    char name[20];

public:
    bool IsHealthy();

public:
    void SetId(int);
    void PrintName();

private:
    float height;
    float weight;

protected:
    …
} ;
```

✓**修饰访问属性关键字可以多次出现**

✓**在声明类时，具有不同访问属性的成员可以按任意顺序出现**

注意事项4—访问属性

```
class  Clock
{
public:
   void SetTime( int h, int m, int s );
   void ShowTime( );
private:
  void ShowTime( );
  int Hour, Minute, Second;
};
```

```
class  Clock
{
public:
   void SetTime( int h, int m, int s );
   void ShowTime( );
private:
  void ShowTime(int format );
  int Hour, Minute, Second;
};
```

✓**一个成员只能具有一种访问属性，否则会出现歧异。**

16

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

3. 对象

对象的概念

✓在C++中，类的定义描述了一个类的所有特征，但类只是一个模板，是抽象的，而一个类的一个**对象**才是一个具体的、具有该类全部特征的实例。

✓类是数据类型，用类定义的变量或实例称为对象，对象分配一定的地址空间，并有相应的属性和行为。

✓同变量一样，对象也是先定义，后使用。

✓对象是类的实例或实体，生成一个类的**对象**，就等于在内存中开辟一块空间，用来保存该类的对象的所有数据，如果一个类有**构造函数**，该类的构造函数也会被调用。

“人”类只有一个，“人”类的实例可以有无数个！

3. 对象 对象定义的语法

类名 对象名；
类名 对象名（构造函数的参数表）；

例：Savings a; *//a是Savings类的对象*
Savings *pa＝&a; *//pa是指向Savings类的指针*
Savings &ra=a; *//ra是Savings类对象a的引用*

✓其中对象名可以是一般对象，也可以是指向对象的指针名或引用名，还可以是数组。

✓对象建立时间：
1. **全局对象**：在主函数开始执行前首先被建立。
2. **局部对象**：在程序执行遇到它们的对象定义时被建立，定义对象时，C++为其分配存储空间。

3. 对象 一般对象及访问方法

一个对象的成员就是该对象的类所定义的成员，有数据成员和成员函数：

✓**一般对象：类名 对象名；**
对象名.成员名 *//数据成员*
对象名.成员函数名（参数表） *//成员函数*

```
void func()
{
    Tdate oneday;              //创建对象
    oneday.Set(2,15,1998);     //调用其public成员函数
    oneday.Print();
}
```

3. 对象 指针对象及访问方法

✓**指针对象：类名 *指针变量名；**
指向运算符：->
指针对象名->成员名 *//数据成员*
指针对象名->成员函数名（参数表） *//成员函数*

```
void someFunc(Tdate *pS)
    {
        pS->Print();   //对象的指针
        if(pS->IsLeapYear())
            cout<< "Yes\n";
        else
            cout<< "No\n";
    }
```

3. 对象 引用对象及访问方法

✓**对象引用：类名 &对象名 = 实际存在的对象名；**
引用名.成员名 *//数据成员*
引用名.成员函数名（参数表）*//成员函数*

```
void someFunc(Tdate &refs)
    {
        refs .Print();  //对象的别名
        if(refs .IsLeapYear())
            cout<< "Yes\n";
        else
            cout<< "No\n";
    }
```

3. 对象 对象成员访问注意事项

1、 ".”用于对象与成员之间，“::”用于类与其成员之间。

2、 对象中的保护（protected）成员和私有（private）成员是不允许被非成员函数直接访问的，这称为类的封装性。

3、类中成员互访，直接使用成员名。

3. 对象 成员函数内部访问成员（this指针）

```
class Counter
{  int value;
   public:
    void setValue( int v )  {
        value=v;
    }
   ……
};
```

```
void main(){
    Counter obj1,obj2;
    obj1.setValue(1);
    obj2.setValue(2);
}
```

- C++为每个非静态成员函数提供一个this指针。
- this指针是一个隐含的指针，它指向了正在被成员函数操作的那个对象。
- this指针不能显式声明，它只是非静态成员函数的一个形参。

◆当通过对象调用成员函数时，编译器会把当前对象的地址赋值给this指针。

3. 对象

成员函数内部访问成员（this指针）

编译器做了特殊处理

```
class Counter{
    int value;
public:
    void setValue( int v ){
        value=v;
    }
    ……
};
```

```
class Counter{
    int value;
public:
    void setValue(Counter *this, int v ){
        this->value=v;
    }
    ……
};
```

```
void main(){
    Counter obj1,obj2;
    obj1.setValue(1);
    obj2.setValue(2);
}
```

```
void main(){
    Counter obj1,obj2;
    obj1.setValue(&obj1,1);
    obj2.setValue(&obj2,2);
}
```

3. 对象

举例：一个完整的Clock类

```
//定义Clock类
class  Clock
{
public:
    void SetTime( int h, int m, int s);
    void addHour(int h);
    void addMinute(int m);
    void addSecond(int s);
    void ShowTime( );
 private:
    int Hour, Minute, Second;
};
```

```
//实现Clock类中的成员函数
void Clock::SetTime( int h, int m, int s )
   {Hour=h;  Minute=m;  Second=s;}
void Clock::addHour( int h )
   { Hour += h; }
void Clock::addMinute( int m )
   { Minute += m; }
void Clock::addSecond( int s )
   { Second += s; }
void Clock::ShowTime( )
   {
    cout<<"Current Time:";
    cout<<Hour<<":"<< Minute<<":"<<
    Second<<endl;
   }
```

28

3. 对象

举例：一个完整的Clock类

```
//使用Clock类
void main( )
{  Clock  clock_1, clock_2;
   clock_1. SetTime( 9 , 5 , 25 );
   clock_2. SetTime( 15 , 16 , 45 );
   clock_1. addHour( 3 );
   clock_2. addMinute( 8 );
   clock_1. ShowTime( );
   clock_2. ShowTime( );
}
```

◈在定义类时，把可以被外部访问的成员说明为public属性，作为外部访问类中成员的接口。把不能被外部访问的成员说明为private或protected属性。

◈ 能调用下面语句吗？

```
clock_1.Hour=21;
```

◈ 如果将ShowTime说明为private属性，还能调用如下语句吗？

```
clock_2.ShowTime();
```

3. 对象

例子：简单int数组类

功能：构造整型数组

```
//数组类的定义
const int ARRAY_SIZE=4;
class MyArray
{
    int data[ARRAY_SIZE];
public:
    int getArraySize();
    int setVal(int pos,int val);
    int getMaxVal();
    int getMinVal();
};
```

```
//数组类的实现
int MyArray::getArraySize()
  { return ARRAY_SIZE;}
int MyArray::setVal(int pos,int val)
  { if((pos<0)||(pos>=ARRAY_SIZE))
      return -1;
    data[pos]=val;
    return 0;    }
int MyArray::getMaxVal()
  { int temp=data[0];
    for(int i=1;i<ARRAY_SIZE;i++)
      {  if(data[i]>temp)
         temp=data[i];}
    return temp;
  }
```

32

3. 对象

例子：简单int数组类

```
//数组类的实现(续)
int MyArray::getMinVal()
{
    int temp=data[0];
    for(int i=1;i<ARRAY_SIZE;i++)
      {
          if(data[i]<temp)
            temp=data[i];
      }
    return temp;
}
```

```
//数组类的使用
int main()
{  MyArray array;
   int i,size,v,max,min;
   size=array.getArraySize();
   cout<<"please input "
         <<size<<" numbers:"<<endl;
   for(i=0;i<size;i++)
     {  cin>>v;
        array.setVal(i,v);  }
   max=array.getMaxVal();
   min=array.getMinVal();
   cout<<"max value is : "<<max<<endl;
   cout<<"min value is : "<<min<<endl;
   return 0;
}
```

33

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

4. 构造函数与析构函数

```
class Desk
{   public:
      int weight;
      int high;
      int width;
      int length;
};

class Stool
{   public:
      int weight;
      int high;
      int width;
      int length;
};
```

```
int main ( )
{
    Desk da; //构造一个桌子对象
    Stool sa; //构造一个凳子对象
    cout <<da.high<<" "<<sa.high; //?
}
```

da	?
	?
	?
	?
sa	?
	?
	?
	?

4. 构造函数与析构函数

◈ 类的数据成员是不能在定义时直接初始化的：

```
class Clock
{
  private:
      int Hour=0, Minute=0, Second=0; //错误
  ……
};
```


4. 构造函数与析构函数

✓1、**类的构造函数**—特殊的成员函数，在用类声明对象时，构造函数将被调用，它通常做一些初始化工作。

✓2、**类的析构函数**—特殊的成员函数，当一个类的对象退出作用域时，它将被调用，它通常做一些扫尾工作。

✓C++的构造函数和析构函数，使类对象能够轻灵地被创建和撤销。

✓**构造函数创建类对象，初始化其成员。**

✓**析构函数撤销类对象。**

✓它们是类的特殊成员函数。

4. 构造函数与析构函数 构造函数、析构函数的名称

✓构造函数和析构函数的名字不能随便起，必须让编译器认出才可被自动执行。 **构造函数、析构函数与类同名，由于析构函数的目的与构造函数相反，就加前缀‘~’以示区别**

Desk类的构造函数名：**Desk ()**

Desk类的析构函数名： **~Desk ()**

✓除了名字以外，构造函数和析构函数**没有返回值类型**，也不允许返回值就像出生和死亡即不带来也不带去。如果有返回值，编译器就不知所措。干脆不要返回值类型。

4. 构造函数与析构函数 构造函数（Constructor）

✓**对象初始化的意义**：对象的意义表达了现实世界的实体，因此一旦建立对象，须有一个有意义的初始值。

✓如果一个类没有专门定义的构造函数，那么C++就仅仅创建对象而不做任何初始化。

✓C++中，建立和初始化对象的过程由**构造函数**来完成，要求建立对象的同时，**自动**调用构造函数，给对象分配空间和初始化。

4. 构造函数与析构函数 Desk类

```
class Desk
{
  public:
     Desk()
     {
       weight = 10;
       high = 5;
       width = 5;
       length = 5;
     }
 protected:
       int weight;
       int high;
       int width;
       int length;
};
```

```
Void fn()
{
    Desk da; //自动调用Desk(),
             //创建对象并初始化
}
```

da	10
	5
	5
	5

4. 构造函数与析构函数

构造函数的定义

✓ *定义方法：*

类名 :: 类名 (参数表)
{ 函数体 }

以类名作为函数名

✓C++规定与类同名的成员函数是构造函数。构造函数的实现也可以在类内、类外。构造函数的特点：

1.**构造函数没有返回类型**
2.**构造函数没有返回值；**
3.**构造函数中有数据成员初始化表。**

➢C++有强类型检查功能，语法错误很容易查出，但是级别高的错误隐藏很深，很难查。

➢有不少的错误是由于变量**没有被正确初始化或清除**造成的，而初始化和清除工作很容易被人遗忘。

➢C++把对象的初始化和清除工作分别放在构造函数和析构函数中。当对象建立时，构造函数自动执行。当对象消亡时，析构函数自动执行。这样就不用担心忘了对象的初始化和清除工作。

4. 构造函数与析构函数

对象数组和类的类成员的构造

```
void fn()
{  Desk dd[5];   //对象数组dd
   Stool sa;
}
```

dd数组创建5个对象，dd的构造函数会被调用5次，从dd[0]开始到dd[4]。

如果一个类的对象是另一个类的数据成员，则在那个类的对象创建所调用的构造函数中，对该成员（对象）自动调用其构造函数。

```
class TutorPair
{ public:
    TutorPair()
      { cout <<"constructing
              tutorpair.\n";
        noMeetings=0;
      }
  protected:
    Student student;
    Teacher teacher;
    int noMeetings;
};
```


4. 构造函数与析构函数

带参数的构造函数

✓无参构造函数的缺点：构造的对象千篇一律，没有个性。

```
class Desk
{   public:
      Desk()
      { weight = 10;
        high = 5;
        width = 5;
        length = 5;
      }
    protected:
        int weight;
        int high;
        int width;
        int length;
};
```

✓带参数的构造函数使初始化一步到位。

```
class Desk
{   public:
      Desk(we,hi,wi,le)
      { weight =we;
        high = hi;
        width = wi;
        length = le;
      }
    protected:
        int weight;
        int high;
        int width;
        int length;
};
```


4. 构造函数与析构函数

初始化数据成员的两种方法

✓方法一：构造函数体内赋值

```
class Desk
{   public:
      Desk(we,hi,wi,le)
      { weight =we;
        high = hi;
        width = wi;
        length = le;  }
    protected:
        int weight;
        int high;
        int width;
        int length;
};
```

✓方法二：带有初始化表的构造函数

```
class Desk
{   public:
      Desk(we,hi,wi,le) :weight( we),
high(hi),width(wi),length(le)
      { }
    protected:
        int weight;
        int high;
        int width;
        int length;
};
```

冒号语法：：

4. 构造函数与析构函数

传给构造函数实参的两种方式

- 如果构造函数带有参数，那么在创建对象时必须给出对应的实参。
- 如果构造函数仅有一个参数：

  className obj = parameter;
- 如果构造函数有一个或多个参数：

  className obj(parameter list);
- 如果构造函数有参数，但是在创建对象时又不给出对应的实参，此时编译出错。

4. 构造函数与析构函数

例：实参传递给构造函数

```
class Clock
{
   private:
     int Hour, Minute, Second;
   public:
     Clock(int h, int m, int s);
       {
         Hour=h;
         Minute=m;
         Second=s;
       }
     ……
};
```

```
class  Counter
{  private:
       int value;
   public:
       Counter(int v):value(v){ }
       ……
};

void main( )
{  Counter counter1(8);
   Counter counter2=9;
   Clock clock1(3, 5, 1);
   Clock clock2( );//错误
}
```

4. 构造函数与析构函数

重载构造函数

◈ 一个类可以提供多个构造函数，但它们的参数表必须不同。重载的目的是为了满足不同的初始化需要。

```
class Clock
{
   private:
        int Hour,Minute, Second;
   public:
        Clock(int h, int m, int s);
        Clock( );
        Clock(char *timestr);
        ……
};
```

```
void main( )
{
   Clock clock1(23, 12, 0);
   Clock clock2( );
   Clock clock3("14:45:32" );
}
```


4. 构造函数与析构函数

具有缺省参数的构造函数

◈ 构造函数也可以有缺省参数。如果在类外实现该函数时，就不能再说明缺省值了。

```
class Clock{
   private:
        int Hour,Minute, Second;
   public:
        Clock(int h = 0, int m = 0, int s = 0);
        Clock(const char *timestr);
        ……
};
```

请思考一下，这个类还能再有一个没有任何参数的构造函数吗？即：Clock();

4. 构造函数与析构函数

默认构造函数

✓1. C++规定，每个类必须有一个构造函数，没有构造函数，就不能创建任何对象。

✓2. 若未提供类的构造函数，则C++提供一个默认的构造函数，该默认构造函数是个无参构造函数，它仅负责构造对象，而不做任何初始化工作。

✓ 3. 只要一个类定义了一个构造函数，C++就不再提供默认构造函数。

✓4. 与变量定义类似，在用默认构造函数创建对象时，如果创建的是全局对象或静态对象，系统将对所有数据成员都初始化为0或空，否则，对象值是随机的 。

```
class Clock
{
   private:
        int Hour;
        int Minute;
        int Second;
   ……
};
```


4. 构造函数与析构函数

析构函数

✓完成对象被删除前的一些清理工作。

✓在对象的生存期结束的时刻系统自动调用它，然后再释放此对象所属的空间。

```
类名 :: ~类名 ()
{ 函数体 }
```

✓析构函数不得返回任何值。

✓析构函数不得带有任何参数。

✓通常为public。

✓调用次序与构造函数相反。

```
Class Student
{public:
  Student()
   {
    Name=new char[20];//分配堆空间
   }
  ~Student()
   {
    Delete [] name; //释放堆空间
   }
 protected:
   char *name;
}
```


4. 构造函数与析构函数

析构函数与构造函数调用次序

```
//**********************
//**    ch12_3.cpp    **
//**********************
#include <iostream.h>
class Student{
public:
  Student()
  { cout <<"constructing student.\n";
    semesHours=100;
    gpa=3.5;}
  ~Student()
  { cout <<"destructing student.\n"; }
protected:
  int semesHours;
  float gpa;
};

class Teacher{
public:
  Teacher()
  { cout <<"constructing teacher.\n";}
  ~Teacher()
  { cout <<"destructing teacher.\n"; }
};
```

```
class TutorPair{
public:
  TutorPair()
  { cout <<"constructing tutorpair.\n";
    noMeetings=0; }
  ~TutorPair()
  { cout <<"destructing tutorpair.\n"; }
protected:
  Student student;
  Teacher teacher;
  int noMeetings;
};

void main()
{  TutorPair tp;
   cout <<"back in main.\n";}
```

析构函数调用次序与构造函数相反：

```
constructing student.
constructing teacher.
constructing tutorpair.
back in main.
destructing tutorpair.
destructing teacher.
destructing student.
```


4. 构造函数与析构函数

析构函数的典型用法

```
class MyString
{ private:
    int len;
    char *buf;
  public:
    MyString(int n);
    void copy(char *src); };

MyString::MyString( int n)
 { len=n;
   buf=new char[n]; }

void MyString::copy(char *src)
 { strcpy(buf,src); }

void main()
 { MyString  obj( 64 );
   obj.copy("helloworld");}
```

！此时，obj的buf所指向的内存空间没有释放

```
class MyString
{ private:
    int len;
    char *buf;
  public:
    MyString(int n);
    ~ MyString( );
    void copy(char *src); };

MyString::MyString( int n)
 { len=n;
   buf=new char[n]; }

void MyString::copy(char *src)
 { strcpy(buf,src); }

MyString::~ MyString( )
{ delete [ ]buf; }

void main()
 { MyString  obj( 64 );
   obj.copy("helloworld");}
```

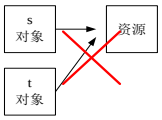
！当obj对象被释放时，析构函数被自动调用，buf所指向的内存空间被释放

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

5. 拷贝构造函数

int a = 5;
int b = a; *//ok*
Student s("Randy", "1234");
Student t=s; *//?*
或 Student t(s); *//?*



✓**拷贝构造函数**是一种特殊的成员函数，它的功能是用一个已知的对象来初始化一个被创建的同类的对象（**克隆**）。拷贝构造函数实际上也是构造函数，它是在初始化时被调用来将一个已知对象的数据成员的值拷贝给正在创建的另一个同类的对象。

55

5. 拷贝构造函数

拷贝构造函数的定义与特点

声明：

拷贝构造函数名（const 类名 & 引用名）；

定义：

类名::拷贝构造函数名（const 类名 & 引用名）
{
函数体
};

其中，拷贝构造函数名是与该类名相同的。const是一个类型修饰符。

1、该函数名同类名，因为它也是一种构造函数，并且该函数也**不能被指定返回类型**
2、该函数只有一个参数，并且是对某个对象的引用
3、每个类都必须有一个拷贝构造函数

56

5. 拷贝构造函数

拷贝构造函数的需要性

✓需要用拷贝构造函数来用一个对象初始化另一个对象的情况：

1、明确表示由一个对象初始化另一个对象
Student t(s); Student t=s;
Student s, t; t = s;

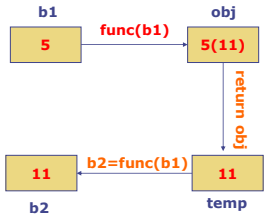
2、当对象作为函数实参传递给函数形参时
int fun(Student s) {…}
fun(stu); *//stu为已声明并初始化的Student类的对象*

3、当对象作为函数返回值时
Student fun(Student & s)
{…; return s;} *//函数返回时要创建临时对象*

✓如果类中没有说明拷贝构造函数，则编译系统自动生成一个默认拷贝构造函数，作为该类的公有成员

57

```
class Counter
{  private:
     int value;
   public:
     Counter(int v)
       {  value=v;  }
     void add(int v)
       {  value+=v; }
     void show( )
       {cout<<value<<endl; }
};
Counter func(Counter obj )
   {  obj.add(6);
      return obj;}
void main( )
   {  Counter b1=5;
      Counter b2=func(b1);
      b1.show();
      b2.show();}
```



◈ 调用**func(b1)**时调用默认的拷贝构造函数
◈ 调用**return obj**时调用默认的拷贝构造函数
◈ 将**func()**返回值赋给**b2**时调用默认的拷贝构造函数

输出：5
11

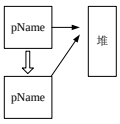
58

5. 拷贝构造函数

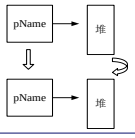
浅拷贝与深拷贝

➢如果不主动编写拷贝构造函数，编译器就将以"**浅拷贝**"的方式自动生成缺省的拷贝构造函数。倘若类中有指针变量就隐含错误。

浅拷贝（位拷贝）：只拷贝成员



深拷贝（值拷贝）：
通过自定义拷贝构造函数不仅拷贝成员，还拷贝资源。




```
Person :: Person(const Person & p)
{  pName=new char[strlen(p.pName)+1];
   if(pName!=0)
       strcpy(pName,p.pName);}
```

59

5. 拷贝构造函数

C++编译器自动产生的四个缺省函数

✓对于任意一个类A，产生如下四个缺省函数：

```
A(void);                    //缺省的无参数构造函数
A(const A &a);              //缺省的拷贝构造函数
~A(void);                   //缺省的析构函数
A & operator = (const A & a);   //缺省的对象赋值函数
```

例：

```
void main()
{ A s, w;   //调用缺省无参构造函数创建对象
  A m(s);   //调用缺省拷贝构造函数创建对象
  m=w;      //调用缺省对象赋值函数将对象w的值赋给对象m
} //调用缺省的析构函数依次析构m、w、s
```


5. 拷贝构造函数

String类

```
class String
{
public:
  String(const char *str = NULL); //构造函数
  String(const String &other); //拷贝构造函数
  ~String(void); //析构函数
  String & operator = (const String &other);//赋值函数
private:
  char *m_data; //用于保存字符串
};
```

```
String::String(const char *str)
{ if(str = =NULL)
    { m_data = new  char[1];
      *m_data = '\0';
    }
  else
    {  int length = strlen(str);
       m_data = new
                 char[length+1];
       strcopy(m_data,str);
    }
}

String::~String(void)
{  delete [] m_data;}
```


5. 拷贝构造函数

String类

```
class String
{public:
  String(const char *str = NULL); //构造函数
  String(const String &other);
        //拷贝构造函数
  ~String(void); //析构函数
  String & operator = (const String &other);//赋值函数
private:
  char *m_data; //用于保存字符串
};
```

```
String::String(const String &other)
{ int length = strlen(other.m_data);
  m_data = new char[length+1];
  strcopy(m_data, other.m_data); }

String & String::operator = (const String &other)
{  if(this == &other) //1.检查自赋值
       return *this;
   delete [] m_data; //2.释放原有的内存
  //分配新的内存资源，并复制内容
   int length = strlen(other.m_data);
   m_data  = new char[length+1];
   strcopy(m_data, other.m_data);
   return *this; //返回本对象的引用
}
```


第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

6. 对象数组

- 与任何其它数据类型一样，可以创建一个类的对象数组。

  Clock clocks[10];

- 通过下标访问数组中的对象，进而访问该对象的公有成员。

  clocks[3].ShowTime();

```
class Point
 { public:
     float x,y;
     Point( ){x=0;y=0;}
     Point(float a){x=a;y=0;}
     Point(float a ,float b)
       {  x=a;y=b;  }
     void show( )
       {cout<<x<<":"<<y<<endl; }
 };

void main( )
{   Point array[3];
    array[1].x=5;
    array[1].y=6;
    array[1].show();}
```

输出：5:6

6. 对象数组

对象数组的初始化

- 对象数组的初始化

  ✓初始化array[0]时调用的是Point(float ,float)构造函数
  ✓初始化array[1]时调用的是Point(float)构造函数
  ✓初始化array[2]时调用的是Point( )构造函数

- 如果在声明数组时给出每一个数组元素的初始值，在初始化过程中就会调用最匹配的构造函数。

```
class Point
{ public:
    Point( )
      {x=0;y=0;}
   Point(float a)
      {x=a;y=0;}
   Point(float a, float b)
      {x=a;y=b;}
   ……
};
void main( )
{  Point  array[3]={ Point(3,4), 5 };
   ……
}
```

6. 对象数组

对象指针的加减操作

```
class Counter
{
   private:
     int value;
   public:
    Counter(int v)
     {  value=v;  }
   void show( )
   {
    cout<<value<<endl;
    }
};
```

```
void main( )
{
   Counter array[3]={5, 6, 7};
   Counter *p=array;
                //或p=&array[0];
   p->show();
   p++;
   p->show();
   p++;
   p->show();
}
```

输出：5
6
7

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

7. 向函数传递对象

- ◈对象本身作为参数
- ◈对象指针作为参数
- ◈对象引用作为参数

7. 向函数传递对象

对象本身作为函数参数

◈由于C++采用传值的方式传递参数，因此使用对象本身作为参数时，形参是实参的一个拷贝。在这种情况下，最好显式地为类定义一个拷贝构造函数，以免出现不容易发现的错误。

```
void main( )
{
    Counter b1=5;
    Counter b2=func(b1);
    b1.show();
    b2.show();
}
```

输出：5
11

```
class Counter
 { private:
     int value;
   public:
     Counter(int v)
        {  value=v;      }
     void add(int v)
        {  value+=v;     }
     void show( )
        { cout<<value<<endl; }
   };

Counter func(Counter obj )
   {   obj.add(6);
       return obj;   }
```


7. 向函数传递对象

对象指针作为函数参数

◈对象指针指向实参对象，通过间接方式访问和修改它所指向的对象，实际上就是访问和修改实参对象

```
void main( )
{
    Counter b1=5;
    b1.show();
    func(&b1);
    b1.show();
}
```

输出：5
11

```
class Counter
 { private:
     int value;
   public:
     Counter(int v)
        {  value=v;      }
     void add(int v)
        {  value+=v;     }
     void show( )
        { cout<<value<<endl; }
   };

~~Counter~~ func(Counter *pobj )
   {   obj->add(6);
       ~~return obj;~~ }
```


7. 向函数传递对象

对象引用作为函数参数

◈这是一种推荐的方式。它比对象本身参数或对象指针参数都要容易理解和使用，同时没有任何的副作用。

```
void main( )
{
    Counter b1=5;
    b1.show();
    func(b1);
    b1.show();
}
```

输出：5
11

```
class Counter
 { private:
     int value;
   public:
     Counter(int v)
        {  value=v;      }
     void add(int v)
        {  value+=v;     }
     void show( )
        { cout<<value<<endl; }
   };

~~Counter~~ func(Counter &obj )
   {   obj.add(6);
       ~~return obj;~~ }
```

7. 向函数传递对象

比较

```
void main( )
{   Counter b1=5;
    b1.show();
    Counter b2=func(b1);
    b2.show();}
```

```
Counter func(Counter obj )
   {  obj.add(6);
      return obj;  }
```

对象本身作为函数参数

```
void main( )
{   Counter b1=5;
    b1.show();
    func(&b1);
    b1.show();}
```

```
void func(Counter *pobj )
   {  obj->add(6);}
```

对象指针作为函数参数

```
void main( )
{   Counter b1=5;
    b1.show();
    func(b1);
    b1.show(); }
```

```
void func(Counter &obj )
   {  obj.add(6);  }
```

对象引用作为函数参数

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

8. 静态成员

为什么需要静态成员？

✓类是类型而不是数据对象，每个类的对象都是该类数据成员的拷贝。如果需要让类的所有对象在类的范围内共享某个数据。声明为static类成员——静态成员

✓一个给定类的每个对象都有类中定义的所有数据成员的拷贝。

✓如果在类中将数据成员用**static**说明为静态的，则这个类成员只有一个拷贝并被这个类的所有对象所共享。

✓年级学生类的数据成员：

- 所属班级
- 年级学生数
- 年级学生平均成绩
- 姓名
- 学号
- 各科考试总成绩

8. 静态成员

类的静态成员定义

✓希望某个类的所有对象都能共享某个成员，就必须将这个成员用关键字**static**定义为静态数据成员：

```
class 类名
{ static 数据类型 数据成员名;
  static 数据类型 成员函数名
        （参数表）
  {
    函数体
  }
};
```

静态数据成员
静态成员函数

```
class Student
{ public:
  Student(char* pName ="no name")
     {…}

  ~Student()
     {…}

  static int number()  //静态成员函数
     {  return noOfStudents; }
protected:
  static int noOfStudents; //静态数据成员
  char name[40];
};
int Student::noOfStudents =0;  //静态数据成员在类外分配空间和初始化
```

8. 静态成员

静态数据成员初始化及访问

✓静态数据成员分配空间初始化必须在类体外进行：

数据类型 类名 :: 静态数据成员名 = 初始值;

✓不同于其它数据成员，在类的外部如果要访问类的静态数据成员可以不通过具体的对象来进行：

Student::noOfStudents ; //ok
Student s;
s.noOfStudents ; //ok

✓静态数据成员是类的成员，而不是对象的成员

```
class Student
{ public:
  Student(char* pName ="no name")
     {…}

  ~Student()
     {…}

  static int number()  //静态成员函数
     {  return noOfStudents; }
protected:
  static int noOfStudents;
  char name[40];
};
int Student::noOfStudents =0;  //静态数据成员在类外分配空间和初始化
```

8. 静态成员

静态成员函数

✓静态成员函数只用于访问静态数据成员。

✓如果静态成员函数中要调用普通成员时，可以通过对象来调用。

✓非静态成员函数有一个指向当前对象的指针this，静态成员函数没有this指针 。因此静态成员函数调用不传递this指针。

✓静态成员函数只与一个类相关， 静态成员函数也不是对象成员，可以不通过对象来调用该类的静态成员函数：

类名::静态成员函数名（参数表）

例：Student::number() ；　　与以下语句效果一样：
Student s；
s.number();

8. 静态成员

例：类的声明部分

```
#include <iostream.h>
#include <string.h>
class Student
{ protected:
    char m_strName[20];
    long m_ID;
  public:
    static int m_nCount;  //静态成员变量，表示已创建对象的数量
    Student(char *strName, long ID); //构造函数
    static int GetCount();          //静态成员函数
    static long GetID(Student);  //对象作为静态成员函数的参数
};
```

注：static int m_nCount在这里设为公有的，以便演示的时候用Student::m_nCount进行访问。

8. 静态成员

例：在类的外部定义类成员函数

```
Student::Student(char *strName, long ID)
{  strcpy(m_strName,strName);
   m_ID = ID;
   m_nCount++;          //对象数目加1
}

int Student::GetCount()
{ return m_nCount; }  //静态成员函数访问静态成员

long Student::GetID(Student x) //通过对象访问非静态成员
{ return x.m_ID;}
```

return m_ID; ✕ //不能直接访问非静态成员m_ID

```
int Student::m_nCount = 0; //在类外初始化静态成员
```


8. 静态成员

例：主函数部分

```
int main()
{  Student e1("LiuMing",1001505);
   //通过类或对象访问静态成员变量：
   cout<<Student::m_nCount<<","<<e1.m_nCount<<'\n';
   //通过类调用静态成员函数：
   cout<<Student::GetCount()<<","<<Student::GetID(e1)<<'\n';
   //通过对象调用静态成员函数：
   cout<<e1.GetCount()<<","<<e1.GetID(e1)<<'\n';
   Student e2("huangli",1001553);
   cout<<Student::GetCount()<<","<<Student::GetID(e2)<<'\n';
   cout<<e2.GetCount()<<","<<e2.GetID(e2)<<'\n';
   //e1和e2共享静态成员变量m_nCount：
   cout<<e1.GetCount()<<","<<e1.GetID(e1)<<'\n';
   return 0;
}
```

```
1，1
1，1001505
1，1001505
2，1001553
2，1001553
2，1001505
```


8. 静态成员

举例：静态成员的特殊应用

- 如果有一个类MyClass，如何设计才能保证在程序运行中该类只能有一个实例？
- 利用静态成员可以解决该问题。

```
#include<iostream.h>
class MyClass
{ private:
    static MyClass *instance;
    MyClass(){} //构造函数
  public:
    static MyClass *getInstance()
      {if(instance==NULL)
        instance=new MyClass();
      return instance;  }
};
```

```
MyClass
   *MyClass::instance=NULL;
void main()
{
   MyClass *obj;
   obj=MyClass::getInstance();
}
```

将构造函数声明为非public属性，可以控制对象的生成！

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

9. 友元

什么是友元

类的友元是一些具有特权的函数或类。它们可以读写特定类的私有和保护类型的数据成员。

✓需要友元的原因：提高了程序的运行效率

✓但是它*破坏*了类的封装性和隐藏性，使得非成员函数可以访问类的私有成员。

✓友元对于运算符重载函数很有用，因为在运算符重载函数中经常需要读取被操作对象数据成员，而数据成员一般不是公有成员。如果将运算符重载函数定义为被运算类的友元，则可以在很大程度上简化编程，节省开销。

✓有友元函数和友元类

9. 友元

友元函数声明与定义

✓在类里声明一个普通函数，标上关键字**friend**，就成了该类的友元，可以访问该类的一切成员。

```
class 类名
{…
  friend 数据类型 函数名（参数表）；//友元函数声明
};
数据类型 函数名（参数表）//友元函数实现
  {函数体}
```

注：参数表中要指明所访问的类，如：类名&
类外定义友元函数的实现不需要加类名

如果一个函数是一个类A的友元，则对于所有以参数形式传递给该函数的A类的对象，都可通过该对象访问所有数据成员和成员函数，而不必理会这些数据成员和成员函数是公有的、保护的还是私有的。

9. 友元

例：使用友元函数计算两点距离

```
#include <iostream.h>
#include <math.h>
class Point
{
 public:        //外部接口
  Point(int xx=0, int yy=0)
    {X=xx;Y=yy;}
 int GetX() {return X;}
 int GetY() {return Y;}
 friend double fDist(Point &a, Point &b);
 private:  //私有数据成员
  int X,Y;
};
```

```
double fDist ( Point& a, Point& b)
{
    double dx=a. GetX() -b. GetX() ;
    double dy=a. GetY() -b. GetY() ;
    return sqrt(dx*dx+dy*dy);
}

int main ( )
{  Point p1(3.0, 5.0), p2(4.0, 6.0);
   double d=fDist(p1, p2);
   cout<<"The distance is "<<d<<endl;
   return 0;
}
```

```
double fDist ( Point& a, Point& b)
{
    double dx=a.X-b.X;
    double dy=a.Y-b.Y;
    return sqrt(dx*dx+dy*dy);
}
```


9. 友元

例：使用友元函数计算两点距离

友元函数的定义与普通函数一样，没有必要指明它在那个类中，这一点很好理解，因为它不是某一个类的成员函数。

友元函数可以直接通过类的对象访问该类的私有成员。

友元函数虽然出现在类定义中，

✓一个类的成员函数可以作为另一类的友元。

友元函数说明可以出现在类的任何地方：在private和protected段均可。

在使用友元函数时，必须在参数表中显式地指出它要访问的对象，在函数原型中当然要指明访问的类。

9. 友元

友元类

✓可以将一个类定义为另一个类的友元类。如果一个类A是类B的友元类，则类A的所有成员函数在对B类对象进行操作时，可以读写B类对象的所有数据成员，包括保护的和私有的数据成员。

✓友元的作用是单向性的：即：如果A类是B类的友元，不等于B类也是A类的友元。此时，B类中的成员函数不能访问A类中的私有数据。

```
class A
 {
    …
 };
class B
{…
 friend class A;
};
```


9. 友元

例：友元类

```
class CAnimal
{public:
   void SetSex(int sex)
     {m_sex = sex;}
   friend class CCat;
 private:
   int m_sex; };

class CCat
{public:
   int GetSex(CAnimal &am)
    {return am.m_sex;}
};
```

```
int main()
{
  CAnimal animal;
  CCat kitty;
  animal.SetSex('M');
  int sex = kitty.GetSex(animal);
  return 0;
}
```

我们可以在CCat类中使用CAnimal 类的private成员变量m_sex。

9. 友元

友元关系总结

◈**友元具有如下性质：**

- 类的友元可以直接访问它的所有成员。
- 友元的声明必须放在类的内部，但放在哪个段没有区别。
- 友元关系不具备对称性，即X是Y的友元，但Y不一定是X的友元。
- 友元关系不具备传递性，即X是Y的友元，Y是Z的友元，但X不一定是Z的友元。

第三章 类与对象

1.结构体与类
2.类的定义
3.对象
4.构造函数与析构函数
5.拷贝构造函数
6.对象数组
7.向函数传递对象
8.静态成员
9.友元
10.类的组合

10. 类的组合

类的组合——一个类的对象作为另一个类的成员，这体现的是整体和部分的关系，即对象的包含关系，这个作为成员的对象被称为子对象。

✓具有无参构造函数的类的子对象创建比较简单。

✓对于具有有参构造函数的类的子对象如何创建？

```
class StudentID
{ public:
    StudentID()          构造函数
      { value=++nextstudentID; }
  protected:
    int value;
};

class Student
{ public:
    Student(char* pName="noName")
      { …… }
  protected:
    char name[20];
    StudentID id;        子对象
};
```


10. 类的组合

✓对于具有有参构造函数的类的子对象如何创建？

✓冒号语法：：

```
class StudentID
{ public:
    StudentID(int id=0)
      { value=id; }
  protected:
    int value;
};

class Student
{ public:
  Student(char* pName="noName",int ssID=0)
    { strcpy(name,pname);
      StudentID id(ssID);   //希望将学号
                            传给学号类对象
                            实际含义为创建
                            一个局部对象,
                            并赋初值ssID
    }
 protected:
    char name[20]
    StudentID id;
};
```

id.value=ssID; ?

StudentID id=9818; ?

或StudentID id(9818); ?

10. 类的组合

冒号语法：解决类中类成员对象初始化

```
class Student
{ public:
  Student(char* pName="no name",int ssID=0):id(ssID)
    { …… }
  protected:
    char name[20];
    StudentID id;
};

void main()
{ Student s("Randy",9818);
  Student t("Jenny");}
```

:id=ssID; // 非法!

若子对象对应的类的构造函数有参数，那么包含该子对象的类必须使用冒号语法初始化表的方式先初始化子对象。

冒号出现在函数参数表之后，含义为在构造函数进入函数体之前就初始化类中的类成员。

10. 类的组合

冒号语法:解决常量和引用初始化

✓冒号语法也使得常量数据成员和引用数据成员的初始化成为可能。

✓冒号语法在派生类的构造时，起到了向基类的构造函数传递参数的作用。——下一章

```
class SillyClass
{ public:
    sillyClass(int &i) :ten(10),refi(i)
      {…}
  protected:
    const int ten; //常量数据成员
    int &refi; //引用数据成员
};

int main()
{
  int i;
  SillyClass sc(i);
}
```

不能这样初始化：
const int ten＝10;
int &refi＝i;

END